

Monocular Depth Estimation With Improved Long-Range Accuracy for UAV Environment Perception

Vlad-Cristian Miclea¹, *Member, IEEE*, and Sergiu Nedevschi², *Member, IEEE*

Abstract—Environment perception by computing the depth is a key task for unmanned aerial vehicle (UAV) type systems. Due to the limited load they can carry, most drones are equipped with a single camera. This prevents general-purpose depth perception methods based either on light detection and ranging (LiDAR) or stereo reconstruction to be effectively used on such platforms. Due to the success of convolutional neural networks (CNNs), monocular depth estimation (MDE) methods have become more and more trustworthy, so their usage on drones is convenient. However, very few such methods have been proposed in the literature, mainly due to the few existing constraints and high diversity that unstructured aerial environments pose. To bridge this gap, we propose a novel approach for MDE, capable to work on aerial images. The method initially proposes an original CNN, particularly adapted to such scenarios. This is done by finding an optimal feature extractor, introducing a new scene understanding module, a new loss and a novel softmax transformation layer that facilitate a better convergence. Furthermore, since both short- and long-range accuracy is required for a robust UAV perception, we introduce a learning-based correction method that redistributes the depth points across the entire depth interval. The proposed CNN gives accurate results, while the additional refinement further improves the accuracy with only a few additional computational resources (around 1–2 ms). We initially show the capabilities of our method on synthetic images captured in unstructured aerial scenarios. Then, we prove that our method can work in real-life situations, computing depth from a single image (at multiple pitch angles) captured by a drone flying in a series of field and forest-like environments. In all these situations, the depth is densely estimated with increased accuracy and reliability.

Index Terms—3-D reconstruction, aerial, convolutional neural network (CNN), interpolation function, monocular depth estimation (MDE), ordinal regression, subpixel, unmanned aerial vehicle (UAV).

I. INTRODUCTION

DEPTH computation is a key task for lots of environment perception applications and benchmarks, such as KITTI [1], Middlebury [2], or NYU [3], facilitating improved

solutions for driving or indoor scenarios. The success of stereo reconstruction [4]–[7], light detection and ranging (LiDAR)-based perception [8], [9], structure from motion [10], or even monocular depth estimation (MDE) [11], [12] in controlled scenarios meant that depth computation for aerial unstructured environments has become an increasingly attainable task. This creates a new challenge for vision tasks, such as aerial-based object detection [13], [14], classification [15], segmentation [16], [17], or recognition [18]. All these methods rely either on LiDAR [15], [16], stereo [19], [20], or even monocular [18] depth sensors to properly perceive the environment.

Stereo reconstruction is the traditional method for depth perception, computing depth similar to human binocular vision. Traditional stereo algorithms have been classified as either local (low accuracy and low cost) or global (reasonable accuracy and high cost). The introduction of convolutional neural networks (CNNs) brought a further increase in accuracy for stereo solutions and supervised end-to-end methods' extracting features, combining them until the network converges toward a specific ground truth (GT). Such methods generally need lots of resources, high computational costs being required for good results. Traditional stereo solutions have been employed for depth perception with reasonable success [21]–[23] in aerial indoor scenarios. The usage of such solutions on a large scale in outdoor environments is limited due to the increased load required by two cameras and their high susceptibility to failures and decalibrations caused by atmospheric conditions (wind, illumination, and so on).

The inherent failures met in practical usage of stereo, in conjunction with the introduction of the disruptive technology of LiDAR, lead to a reduction in use for camera-based depth perception solutions. Although very accurate for mid-range measurements, LiDAR has several disadvantages: it is more expensive and heavy, it gives sparse results, its results are provided in a different coordinate system than the image frame, and the long acquisition period makes it hard to be synchronized with other sensors. Although these problems are (more or less) solvable for simple scenarios, they make LiDAR-based 3-D perception too cumbersome [24], [25] for drone applications since they assume a higher degree of freedom.

With the increased prosperity of deep learning, CNNs have been also used for depth computation from a single image. Such methods belong to the task of MDE [11], [12].

Manuscript received August 14, 2020; revised January 3, 2021; accepted January 12, 2021. Date of publication March 8, 2021; date of current version December 9, 2021. This work was supported by the Romanian Ministry of Education and Research, through the CNCS UEFISCDI Grant, Integrated Semantic Visual Perception and Control for Autonomous Systems (SEPCA), code PN-III-P4-ID-PCCF-2016-0180, under Grant 9/2018. (Corresponding author: Sergiu Nedevschi.)

The authors are with the Department of Computer Science, Technical University of Cluj-Napoca, 400114 Cluj-Napoca, Romania (e-mail: vlad.miclea@cs.utcluj.ro; sergiu.nedevschi@cs.utcluj.ro).

Digital Object Identifier 10.1109/TGRS.2021.3060513

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

MDE methods have several advantages over their counterparts (and this is especially relevant for drone applications): the solution is cheaper and lighter in terms of both equipment and computational resources, there is no need for extra temporal alignment, and it produces results with reasonable accuracy. The main problem with depth perception from a single camera comes from its ill-posed formulation: there is infinity on 3-D maps that can be generated from a 2-D image. MDE solutions solve this problem by learning, including additional cues to aid the process. The complexity of unstructured aerial environments makes simple learning mechanisms too weak for convergence toward a trustworthy depth map, requiring additional support. Furthermore, aerial images generally assume a larger range of depth measurements, so both near- and far-range accuracy improvements are required.

We propose here a method that tries to overcome the inherent weaknesses that the MDE method has in aerial scenarios. We do this by aiding the deep learning process with more complex layers that include additional cues, constraints, and learning mechanisms such that a better depth map is computed for images captured from a drone. Thus, we find an optimal feature extractor, generate a novel upsampling subnetwork with the role of scene understanding, and propose a new loss, particularly adapted to aerial images. Since the best results in the literature are obtained when the MDE network is modeled as a (form of) classifier, we propose a multitask loss function that incorporates: 1) a regular classification loss and 2) an ordinal classification (ordinal regression) loss. In order to apply the two losses over the same softmax volume, we introduce a novel transformation layer (called *sf_trans*) that mathematically modifies the softmax layer required by the ordinal regression to a softmax usable by a regular classifier.

Furthermore, since classification produces results in a discrete limited range, we refine the depth by correcting the inherent error caused by the classification. Toward this goal, we introduce a learning procedure that generates an interpolation function by using softmax probabilities at neighboring depth values. The procedure uses genetic algorithms (GAs) as means of optimization, and it resembles methods used for subpixel postprocessing. To sum up, our main novelties are as follows.

- 1) We present a novel CNN that leads to accurate MDE results by modeling the problem as a (form of) classification; we highlight the design decisions possible for such a model (feature extractors, prediction layers, and losses).
- 2) We introduce a novel loss that is suitable for MDE from aerial images modeling the problem as an ordinal regression and as a regular classification.
- 3) We propose a novel differentiable layer representing a mathematical transformation module that translates the ordinal regression-based softmax probability volume to a classification-based vector.
- 4) We propose a novel subpixel refinement method that proves to redistribute points across the entire depth domain, by training a GA to generate an interpolation function.

- 5) We show that our method obtains good results for aerial synthetic images captured in unstructured aerial domains (in a forest).
- 6) We prove that our method can work on a large range of real-life images captured from a drone flying in field- and forest-like environments.

This article starts with presenting the state of the art in camera-based depth estimation, highlighting the main approaches for unmanned aerial vehicle (UAV) depth perception. It continues with presenting the design decisions that lead to a good CNN for MDE and highlight the layers required for the CNN development. Section II introduces a new depth refinement strategy and the GA that corrects the depth map generated by the CNN. In Sections V and VI, we present a thorough evaluation of our method on both real and synthetic aerial images, and we discuss the improvements given by our method. We conclude this article in Section VII.

II. RELATED WORK

A. Camera-Based Depth Perception for UAV Systems

Although depth estimation is a much researched domain in the case of ground-related scenarios (for driving or indoor) [11], [12], [26], [27], depth estimation for aerial environments is still a new task, with few proposed solutions.

Several traditional depth computation methods based on stereo reconstruction [21], [22], [28] exist in this domain obtaining decent results for images captured in perfect conditions in controlled/indoor environments. However, due to the possible decalibrations of the camera (mostly caused by the atmospheric conditions) or the high load that cannot be carried by the drone, such approaches are less usable. Other approaches based on LiDAR [15], [16], [29], [30] or structure from motion [18], [31] are less efficient either because of the sensor alignment (in the first case) and the sparsity of the results (both cases).

Learning-based solutions are also scarce. This is not only due to the lack of sufficient training data required for accurate results but also due to the lack of structure that aerial images presume (too many degrees of freedom for the drone). There are several attempts toward obtaining an accurate depth map of outside environments from low- [32], [33] and far-range [34], [35] aerial images. Most of these novel attempts try to create a data set by using synthetic images generated through a simulator. Several such systems (e.g., [36] and [37]) are capable to produce highly confident depth GTs, but the synthetic RGB images are generally unrealistic (they present problems with illumination and textures), which leads to low-quality results.

B. Monocular Depth Estimation

Since an infinity of 2-D depth maps can be generated from a 3-D scene, MDE approaches try first to understand locally the relations between objects and then exploit these cues [11]. Learning the scene representation for this requires CNNs. Most of the approaches in the MDE category use encoder-decoder architectures, minimizing a mean squared error (MSE)-type of loss [38], [39]. The most representative ways to capture object relation representation are by either extracting features

at multiple scales (under the form of a pyramid) [40], by using dilated convolutions [41], or by using attentional gating [42] (features from initial layers (coarse) being later plugged in for better filtering of relevant information).

Although most of the MDE algorithms directly regress the (fractional) depth map, Li *et al.* [43] proved that better results are obtained when the MDE problem is seen as a classification rather than a regression [43, Tables 3 and 4]. In the classification case, the network converges faster, and it produces more robust results. However, classification assumes a fixed number of depth classes, meaning that the depth interval has to be discretized. A common way to discretize the spatial depth interval is to use uniform discretization (UD) [26], [44], which equally accounts for the near-side classes and the far-side ones.

DORN [27] proposes an improved discretization technique, using the log-space (more classes in the near-space). Such an approach is capable to more accurately predict short-range and near-range depth values, which constitutes the vast majority of points inside the data set (because of this DORN ranked first for more than one year on Kitti). Another important factor for the success of DORN is that its loss also accounts for neighboring classes. Such a loss is based on ordinal regression (or ordinal classification), and it provides higher accuracy and faster convergence. There are other similar approaches (such as SORD [45]) that also perform very well by the understanding of the intraclass and interclass ordinal relationships.

Due to the lack of sufficient training data, supervised MDE methods generally perform worse than stereo counterparts. In order to alleviate this, semisupervised [46], [47] and unsupervised [48] learning-based MDE approaches have emerged. These methods combine the depth prediction of two supervised networks (for left and right) with their corresponding stereo-based disparity information in a single loss, capable to produce a better depth map. Other works, such as [49] and [50], use self-supervising techniques by processing a single image to obtain the second synthetic view (similar to the right view in stereo). A CNN is then trained on the two images for computing a disparity map.

C. Depth Perception at Large Distances

The problem of depth perception at large distances is very important for outside aerial scenarios. The problem has been studied in the context of stereo reconstruction, mostly in approaches related to driving applications. In their study, Pinggera *et al.* [51] show that metric distance errors increase nonlinearly for given stereo disparity errors. For instance, a stereo algorithm with a 1/2 pixel error will lead to an error of 20 m when trying to measure an object at 150 m. Therefore, computing the fractional part of the disparity becomes extremely important. Due to their formulation as similar to stereo, MDE methods suffer from the same problem.

From the perspective of range, one can classify camera-based depth perception algorithms as either discrete or continuous [52]. According to these criteria, algorithms in continuous space have proven more accurate with increased density, at a larger computational cost. For such algorithms,

fractional disparities are either directly inferred from the optimization [51] or mathematically derive an optimal function that compensates for the error introduced by the cost computation metric [53].

On the other hand, discrete algorithms offer real-time performance, with pixelwise comparable results. However, when subpixel accurate results are required (for long-range accuracy), they suffer from the *pixel-locking* effect: overcrowding of fractional depth values toward integer values. Generally, pixel-locking is solved by inferring a function based on values extracted from the cost volume at the chosen depth [54], [55]. There are several functions that generally seem to provide good results: parabola [56], equiangular [57], or SinFit [58].

III. CONVOLUTIONAL NEURAL NETWORK

This section initially introduces the CNN, and it presents its components: the feature extractor, the upsampling module, the softmax layers, and possible losses. We will also highlight the novel layer that mathematically transforms probabilities given by softmax in the case of ordinal regression to probabilities useful in the case of classification. The overall architecture of the system is presented in Fig. 1, while the CNN is detailed in Fig. 2. The network consists of several parts.

- 1) A dense feature extractor that extracts data at multiple scales while preserving the pixel-level structure.
- 2) An upsampling module that combines the aforementioned extracted information.
- 3) A softmax layer that assigns a probability to each pixel to belong to a specific depth class, building a 3-D volume.
- 4) A softmax transformation layer that modifies the probabilities for a regular classification loss.
- 5) A novel loss that includes both an ordinal classification (ordinal regression) and a regular classification counterpart. This loss is customized such that it better deals with aerial images.

A. Feature Extraction Module

For this part, before we focus on a specific technique, we initially specify some basic guidelines for a good feature extractor for depth estimation. First, while several methods use pooling layers, these proved to be very inefficient for depth estimation [11]. The main reason for this is the observation that pooling layers lead to a loss of pixel interconnection information. Second, we need features at multiple scales in order to properly capture the object ordering. For this reason, we focus on feature extractors that employ skip connections instead of pooling. This structure information is kept, while the network also captures the context at different scales for object ordering context. According to these guidelines, several feature extraction approaches (generally introduced in the context of object detection) can be valuable.

- 1) *Resnet* [59]: This is the first model that introduced skip connections as means to preserve the scene structure while capturing the relevant information at various scales. Depending on the maximum feature depth, there are multiple variants for this network. Resnet-101 is

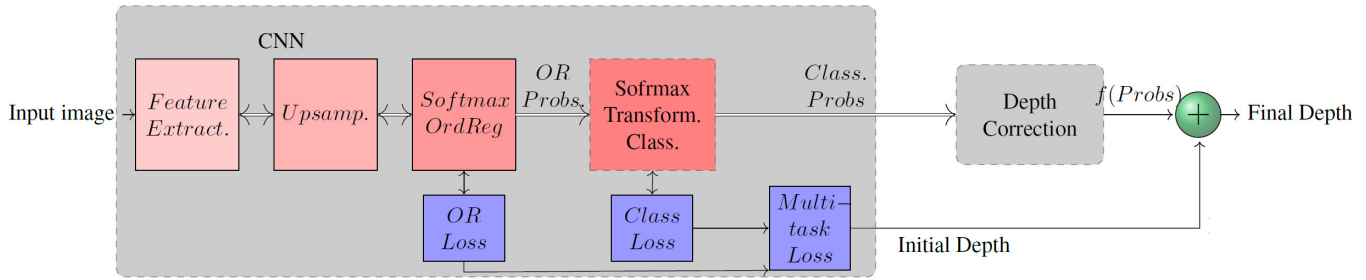


Fig. 1. Architecture of the system.

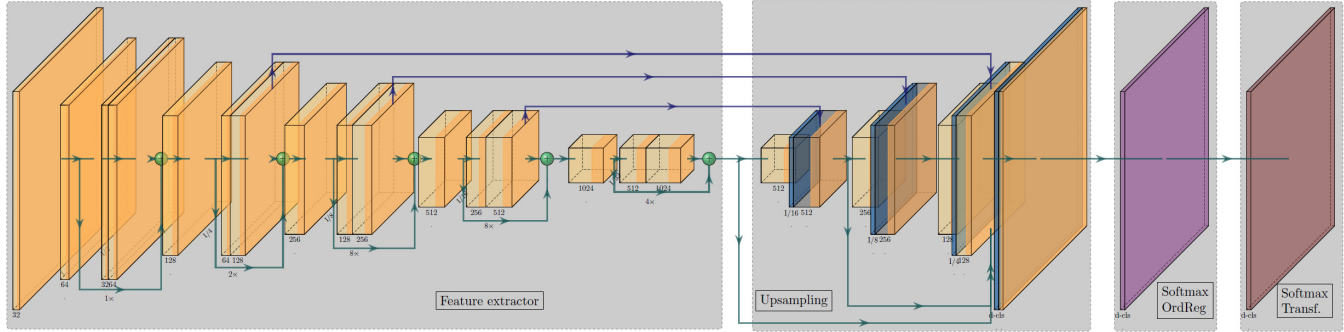


Fig. 2. Proposed MDE architecture for aerial images.

generally considered the most robust feature extractor in this category, being also used for several MDE solutions [27].

- 2) Inception [60] is the feature extractor used in GoogleNet, and it also acts as a multilevel feature extractor, by computing 1×1 , 3×3 , and 5×5 convolutions within the same module of the network. The weight size for Inception is smaller than ResNet, but its accuracy is also worse.
- 3) Xception [61] is a modified version of the Inception architecture. The main difference with respect to Inception is that it replaces the standard modules with depth-wise separable convolutions, which leads to a smaller model.
- 4) Darknet is a lightweight feature extractor, initially introduced in the Yolo [62], proving that the scene can be understood very accurately by using fewer training parameters. Among all the proposed variants, the third generation of such models (feature extractor is called Darknet-53) is the most complex and represents the best tradeoff between speed and accuracy.

From the above list of options, we choose Darknet-53. The method requires only half of the ResNet-101 parameters while obtaining similar accuracy. Thus, our feature extractor subnetwork is modeled mostly by 3×3 and 1×1 convolutional layers. Skip connections are employed to preserve the image structure through all 53 layers. The resolution is decreased after each residual link, the number of features being doubled. The output of the feature encoder consists of four types of features: 1024 features of size $I/32$, 512 features at size $I/16$, 256 features at size $I/8$, and 128 features at size $I/4$, where I is the image size. The feature extraction subnetwork is depicted in Fig. 2 (leftmost block).

B. Upsampling Module

All the previously mentioned feature extractions make predictions at multiple scales, the features being extracted in a feature pyramid-like encoder. Predictions at multiple scales are required due to the ill-posed formulation of MDE. Thus, small-scale features capture local information, while skip connections and large-scale features ensure global context.

Once features are extracted, the upsampling part cleverly combines them, using the aforementioned features at multiple scales to understand the scene at various levels. In order to do this in a way that does not increase the memory footprint, instead of deconvolutions (heavy computations), we restrict to simple convolutions, dilated convolutions, and upsampling layers. The features used for detection, at $I/32$, $I/16$, $I/8$, and $I/4$ sizes, are, therefore, combined with features extracted at the same scales from the feature extractor. Finally, the output is upsampled four times (followed by a final convolution), to match the input resolution, and we build a 3-D volume to match the (following) softmax layer, which is modeled in relation to the chosen loss. The upsampling subnetwork is depicted in Fig. 2 (second block).

C. Learning Using Ordinal Regression

In order to properly model the learning, we initially use the remarks presented by Li *et al.* [43], which shows that better results for depth estimation are obtained when this task is seen as a classification, rather than a simple linear regression. Furthermore, state-of-the-art benchmarks show that ordinal regression (also known as ordinal classification) outperforms simple classification counterparts, approaches, such as DORN [27] or SORD [45], being very accurate. Ordinal

regression can be seen as a more complex classification, in which values from neighboring depths are accounted for in the final loss. This has the effect of both finding the optimal depth class value for each pixel and also an effect of smoothing since neighboring classes will have an influence over the final chosen depth.

Thus, we initially follow these guidelines, modeling the problem as a pure ordinal regression. We used the loss proposed in DORN [27], which is one of the top approaches in the Kitti [1] benchmark. The loss for each pixel is computed as

$$L_{\text{OR_Depth}} = - \left(\sum_{d=1}^{\text{depth_GT}-1} \log(C_{\text{sf_or}}(p, d)) + \sum_{d=\text{depth_GT}}^{\text{depth_classes}} \log(1 - C_{\text{sf_or}}(p, d)) \right) \quad (1)$$

where depth_GT is the GT depth value, p is the current pixel, and the probability given by the softmax function $C_{\text{sf_or}}$ will thoroughly be discussed in the following. The overall loss will be the sum of losses for each pixel. The number of classes depth_classes will be directly dependent on the depth interval discretization, and it can be chosen by considering

$$\text{depth_classes} = \frac{\text{max_depth}}{\text{samp_f}} \quad (2)$$

where max_depth is the maximum depth allowed for measurement (e.g., 80, for driving applications) and samp_f is the sampling factor for the discretization (how large is a depth interval). For instance, if $\text{samp_f} = 1$ and the depth space is uniformly discretized, then all the integer metric distances are considered as possibilities.

Although such a model proves to work very well on driving [1] or indoor [3] benchmarks, the learning modeling mostly assumes a predictable ordering of objects in the scene, depths smoothly changing. In comparison to cars and indoor robots, UAVs generally have a larger depth range and a larger degree of freedom (e.g., the ground is not always in front), which means that this smoothness assumption does not necessarily apply in the case of aerial scenarios. We exhaustively evaluated the ordinal regression loss, and we observed that the method does not properly converge when dealing with aerial images. First, the smoothness assumption creates problems around the edges of the objects. Second, the ordinal regression loss does not properly capture the *others* class (there are lots of points beyond the predefined max_depth cutoff). This is required for our scene since we cannot predict depths for the entire depth interval captured from the air. We will properly discuss all these shortcomings in the evaluation section.

An alternative way to model the problem is by using a regular classifier as learning, with cross-entropy as loss

$$L_{\text{Class_Depth}} = - \sum_{d=1}^{\text{depth_classes}} y(p, d) \log(C_{\text{sf_cls}}(p, d)) \quad (3)$$

where $C_{\text{sf_cls}}$ is the probability that a pixel p belongs to class d (given by the classification-based softmax function), while y is the GT value, and it is 1 for the correct class and 0

otherwise. Although this type of modeling might help with the aforementioned problems, it also causes other sorts of issues, such as lots of false positives and irregular distribution of depths.

Thus, ideally, we would have the benefits of both losses, by complementing each other. A simple and well-known way to do this is by introducing two separate branches, with two different types of probabilities given by two separate softmax layers. On the other hand, since both models lead to a classification, using the same features, a two-branch model will be wasteful in terms of resources. A compact model will use a single softmax layer, which can enable both losses simultaneously. Unfortunately, such modeling is not straightforward since the probabilities in the two cases prove to be differently distributed. Thus, a series of transformations are required for the network to work.

D. *SF_trans*—A Novel Softmax Transformation Layer

The softmax probability function in the case of ordinal regression ($C_{\text{sf_or}}(p, d)$) [27, eq. (3)] looks similar to a form of a cumulative distribution function

$$C_{\text{sf_or}}(p, d) = \frac{e^{C_{\text{conv}}(p, 2d)}}{e^{C_{\text{conv}}(p, 2d)} + e^{C_{\text{conv}}(p, 2d+1)}} \quad (4)$$

where C_{conv} represents the 3-D feature vectors given by the last convolution in the upsampling subnetwork. More clearly, we can observe that the distribution for a given spatial location p varies according to the inverse logistic function [see Fig. 3 (top)], where the vertical line highlights the position of the optimal (chosen) depth. In this case, the best depth value is not selected by finding the position of the maximum probability from this distribution (as in classification) but rather by finding the median value of the function. Thus, for ordinal regression, the chosen depth will be given by the position of the last value whose probability is not smaller than 0.5 (last class in which the probability is still closer to 1)

$$\text{dep}_{\text{OR}}(p) = \arg_d(C_{\text{sf_or}}(p, d) \geq 0.5). \quad (5)$$

Mathematically, this logistic function is continuous and differentiable on interval $[0, \dots, \text{depth_classes}]$. Our goal is to transform this distribution such that the same position is given, but its form can be exploited by the cross-entropy loss [similar to Fig. 3 (bottom)]. Thus, in order to obtain such a distribution, we will have to mathematically transform $C_{\text{sf_or}}(p, d)$ to a new curve distribution $C_{\text{sf_cls}}(p, d)$. In order to properly work in CNN environments, backpropagation has to be enabled, so the new distribution will have the following properties.

- 1) $C_{\text{sf_cls}}(p, d)$ is differentiable on the interval $[0, \text{depth_classes}]$.
- 2) The minimum position for $C_{\text{sf_cls}}(p, d)$ is dep , i.e., $C_{\text{sf_cls}}(p, \text{dep}) \leq C_{\text{sf_cls}}(p, d)$, $\forall d$. Ideally, $C_{\text{sf_cls}}(p, \text{dep}) \approx 0$.

By considering these two properties, we propose the novel softmax transformation layer that is described by the function

$$C_{\text{sf_cls}}(p, d) = 1 - [C_{\text{sf_or}}(p, d) - C_{\text{sf_or}}(p, \text{dep})]^2. \quad (6)$$

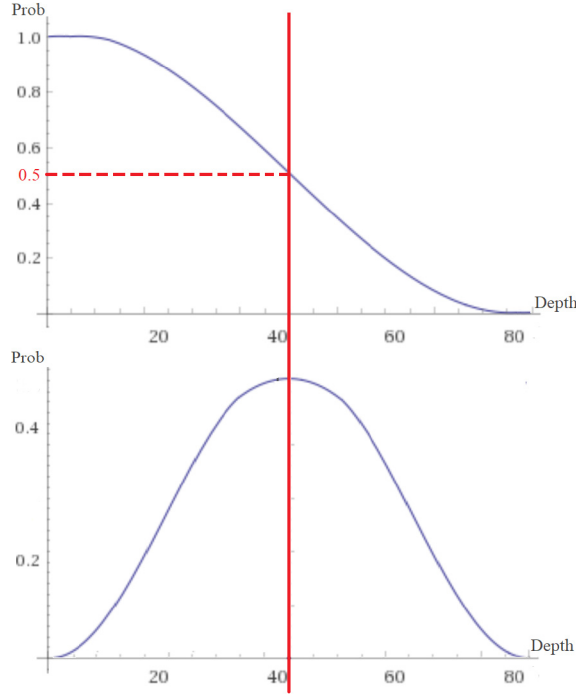


Fig. 3. Probability distributions for ordinal regression softmax $C_{sf_or}(p)$ (Top) and ordinal classification softmax $C_{sf_cls}(p)$ (Bottom) for a specific pixel; The position of the minimal probability is highlighted.

We can easily prove that our newly generated distribution curve $C_{sf_cls}(p, d)$ follows the aforementioned properties. First, if $d \rightarrow \text{dep} \implies C_{sf_cls}(p, d) \rightarrow 0$. For the backpropagation, we are interested in the form of the new function. This can be seen if we expand and then derive $C_{sf_cls}(p, d)$ with respect to d

$$C_{sf_cls}(p, d) = 1 - (C_{sf_or}^2(p, d) - 2C_{sf_or}(p, d) \times C_{sf_or}(p, \text{dep}) + C_{sf_or}^2(p, \text{dep})) \quad (7)$$

$$\begin{aligned} C'_{sf_cls}(p, d) &= -2C_{sf_or}(p, d)C'_{sf_or}(p, d) \\ &\quad + 2C'_{sf_or}(p, d)C_{sf_or}(p, \text{dep}) \\ &= -2C'_{sf_or}(p, d)[C_{sf_or}(p, d) \\ &\quad - C_{sf_or}(p, \text{dep})]. \end{aligned} \quad (8)$$

Approximating the derivative around dep gives that

$$C'_{sf_cls}(p, \text{dep}) \approx -2C_{sf_or}'^2(p, \text{dep}). \quad (9)$$

As mentioned, the minimum point for $C_{sf_cls}(p, d)$ will be dep . The second observation that we can draw from the previous expression is that our newly generated distribution will increase/decrease quadratically with respect to the original distribution. Thus, a larger interclass difference is introduced, which is critical for unstructured environments.

E. Multitask Loss

Once the probability transformation function is enabled, the final loss can be introduced. Thus, the multitask loss that combines both ordinal classification and regular classification will be

$$L_{\text{Depth}} = L_{\text{OR_Depth}} + \alpha \times L_{\text{Class_Depth}} \quad (10)$$

where α is a tuning parameter, which is linearly modified with respect to the iteration number in training. In terms of convergence, we initially train only using ordinal regression, until the depth is smooth enough (for around 20–30 epochs). Then, we gradually increase the classification loss, which further refines the depth map, providing improved edges and corners.

For the next steps in the workflow (postprocessing), at evaluation, the CNN will output two maps:

- 1) An initial 2-D depth map that converged from the CNN;
- 2) A 3-D probability volume that constitutes the values given by C_{sf_cls} softmax function (after transformation).

IV. DEPTH CORRECTION FOR LONG-RANGE PERCEPTION

Once the CNN is trained, the final depth map will be further postprocessed, such that a larger range of values is obtained.

A. Fine-Discretization of the Space Versus Interpolation

Our softmax layer produces only a subset of depth values for each pixel, corresponding to the *depth_cls* depths. The number of classes directly depends on the discretization of the depth interval given by the GTs. For instance, Kitty [63] and Cityscapes [64] use 16 bits to embed their GT depth data, which results in 65 536 depth possibilities. A large number of classes entails a large “softmax probability” volume (we can name it also a “cost volume” due to the similarity to the 3-D volume from stereo cost computation [65]), which affects the footprint of the GPU memory and, thus, the overall speed of the CNN.

A common way to discretize the spatial depth interval $[a, b]$ is to use UD [see Fig. 4 (top)]. In such an approach, depth intervals are equally distributed along with the entire space. DORN [27] proposes an improved discretization, using the log-space [see Fig. 4 (middle)]. Such an approach is capable of more accurately predict small and medium depth values, which constitutes the vast majority of points inside the data set.

Although DORN performs extremely well on benchmarks, such as Kitty [63], the results that it produces are still quite inaccurate, especially for objects at large distances. Unlike driving environments, in which around 90% of the points are in front, the proportion is far smaller on aerial images. Therefore, rather than only focusing on smart ways to discretize the depth interval, we adopt a different approach. We initially compute a depth map using only a small number of classes (fewer depth intervals). We then interpolate the correct value: the *fine* corrected depth should slightly drift from the chosen depth class. Our approach resembles subpixel disparity correction methods, previously proposed in stereo reconstruction.

B. Using Neighbor Depth Values

As previously mentioned, the softmax transformation layer in the CNN produces a *width* $\times$ *height* $\times$ *depth_classes* probability volume, assigning a probability for each position to belong to a specific depth class. During the evaluation phase,

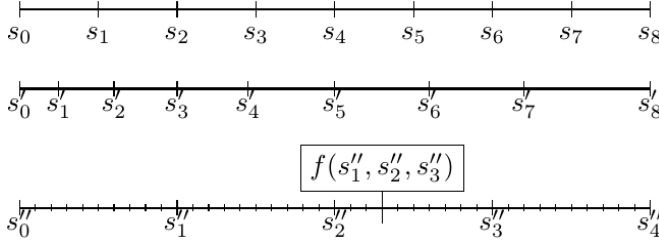


Fig. 4. Discretization techniques (Top and Middle) versus interpolation (Bottom).

the chosen depth (coarse) is selected from the transformed volume by

$$\text{dep}_{\text{coarse}}(p) = \text{argmax}_d(C_{\text{sf_cls}}(p, d)). \quad (11)$$

For a finer depth map, the selected coarse value dep must be corrected. By definition (in the case of ordinal regression) and also by exhaustive testing (in the case of simple classification), probabilities neighboring the chosen class are the ones that should support the correction. This is really important because it shows that the final depth value should be slightly modified and the neighboring probabilities should hint the direction in which the depth should drift. Thus, we adapt the coarse value, such as

$$\text{dep}_{\text{fine}}(p) = \text{dep}_{\text{coarse}}(p) + f[C_{\text{sf_cls}}(p, \text{dep} - 1), \\ \times C_{\text{sf_cls}}(p, \text{dep}), C_{\text{sf_cls}}(p, \text{dep} + 1)] \quad (12)$$

where $C_{\text{sf_cls}}(p, \text{dep})$ is the softmax probability at the chosen depth, while $C_{\text{sf_cls}}(p, \text{dep} - 1)$ and $C_{\text{sf_cls}}(p, \text{dep} + 1)$ represent the neighboring probabilities (at next and previous depth classes). The most important part of this is finding the function f , which should be a function that redirects the depth toward the correct value.

This function is heavily constrained, having several properties. First, it should be symmetrical with respect to the chosen coarse depth value c_d (the same drift should be allowed on both directions). Two examples of probabilities alternations are presented in Fig. 5. In the left case [see Fig. 5(a)], the probability for both neighbors $\text{dep} - 1$ and $\text{dep} + 1$ has similar values (and much more smaller than the probability at dep). Thus, in this case, the final value should be exactly the coarse value dep . Fig. 5(b) shows the opposite case, in which one of the two neighboring probabilities [e.g., $C_{\text{sf_cls}}(\text{dep} - 1)$] has a value really close to $C_{\text{sf_cls}}(\text{dep})$. In such a case, the true (fine) depth value should rely between $\text{dep} - 1$ and dep . These two examples (the most relevant boundary cases) give us a hint about the properties of function f .

Mathematically, we can insert these properties in the following way. First, without loss of generality, we can translate all the probabilities to the origin of c_d on the real axis such that

$$l_d = C_{\text{sf_cls}}(\text{dep} - 1) - C_{\text{sf_cls}}(\text{dep}) \\ r_d = C_{\text{sf_cls}}(\text{dep} + 1) - C_{\text{sf_cls}}(\text{dep}). \quad (13)$$

Furthermore, due to the same symmetrical property, we can further reduce the complexity of the formulation by embedding

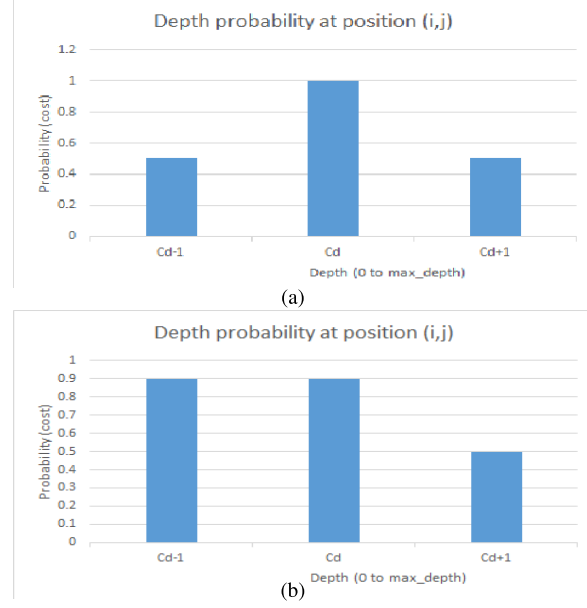


Fig. 5. Cases for softmax probabilities in the 3-D volume. (a) Max difference between probabilities. (b) Min difference between probabilities.

both probabilities into the same variable x (or on its inverse, if the symmetrical case is considered). Thus

$$f(x) = f(l_d/r_d). \quad (14)$$

Thus, we can reformulate the depth value computation (12), by considering the observations and properties (13) and (14). We get

$$\text{dep}_{\text{fine}} = \begin{cases} \text{dep}_{\text{coarse}} - \text{samp_f}/2 \times (-1 + f(x)) & \text{if } l_d \leq r_d \\ \text{dep}_{\text{coarse}} + \text{samp_f}/2 \times (1 - f(1/x)) & \text{otherwise} \end{cases} \quad (15)$$

where samp_f is the distance chosen for the interval between two successive coarse values, and it is a predefined value [also defined in (2)].

For instance, if the sampling factor is 1 (integer values are considered as coarse depth values), (15) becomes

$$\text{dep}_{\text{fine}} = \begin{cases} \text{dep}_{\text{coarse}} - 0.5 + 0.5 \times f(x) & \text{if } l_d \leq r_d \\ \text{dep}_{\text{coarse}} + 0.5 - 0.5 \times f(1/x) & \text{otherwise.} \end{cases} \quad (16)$$

To sum up, we showed that we can further improve the depth value if we know the probabilities neighboring the chosen depth class. The goal is to find the optimal correction function $f(x)$ such that the depth is properly drifted toward the desired value.

C. Function Fitting

Our main goal here is to find the proper function that correctly redistributes the points to the desired fractional depth value. Any function that takes values in the interval $[0, \text{samp_f}/2]$ and gives values in the same interval can be generally used. However, only a small set of functions are

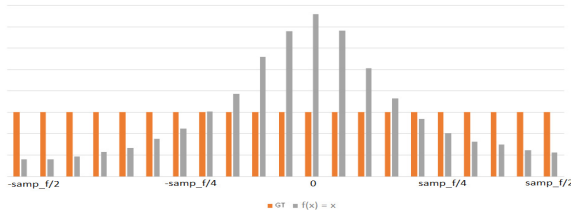


Fig. 6. Fine depth histogram: GT versus interpolation function.

effective, and they can be found by learning mechanisms. For this, we focus on the methodology of function fitting.

A clear understanding of the redistribution behavior for a particular interpolation function can be seen by shifting the view from 3-D space to a histogram representation. Let us consider the histogram classes (bins containing the fractional values) as being the subclasses of points corresponding to equal depth subintervals in the range $[-\text{samp}_f/2, \text{samp}_f/2]$. Each image pixel from the scene is classified according to its fractional part of the depth in one of the histogram bins. In Fig. 6, we show a histogram representation of points that resulted after applying $f(x) = x$ function with respect to the GT. The GT points are randomly chosen, but, for representation purposes, we force an equal number of points for all GT classes. We can observe that the distributions are not similar. A very good function will distribute the points in the same classes as the GT.

D. Genetic Algorithm for Function Optimization

The easiest way to create a valuable function is by learning. A learnable way to represent the function is to regard it as a binary expression tree. We then use a stochastic optimization strategy based on GAs to evolve the optimal function by continually modifying the branches in the tree. For generating the expression tree, we will use unary operators: (sin, cos, tan, arcsin, arccos, arctan, log, and sqrt) and binary operators: +, −, /, and * as tree nodes. In the case of operands, this will constitute the leaves of the trees, and we allow for constants (1, 2, 5, π , and e) and variables (x).

The stochastic optimization method consists of a GA that will initially start from a set of functions (expression trees), and it will iteratively improve them by means of the following:

- 1) selection (keeping only the best functions from a generation);
- 2) crossover (interchange branches between the best expression trees);
- 3) mutation (randomly changing nodes inside the tree).

In order to ensure convergence, we must provide a fitness function. For this, we use the following error function:

$$f_{\text{fitness}} = \sum_{c=1}^{\text{class}} |\widetilde{f(i, c)} - GT(c)| \quad (17)$$

where $\widetilde{f(i, c)}$ is the median value among the functions and class corresponds to the number of classes when discretizing interval $[-\text{samp}_f/2, \dots, \text{samp}_f/2]$. We train on a randomly chosen subset of points obtained from the entire data

set, ensuring that the number of points inside each class is constant. The best function we obtain is

$$f(x) = \sin(\arctan(x \times \pi/2) \times \pi/2). \quad (18)$$

The function is bijective, taking values from the interval $[0, \dots, \text{samp}_f/2]$, giving values in the same interval and, thus, distributing the points to the desired depth.

V. EVALUATION USING AERIAL SYNTHETIC IMAGES

A. Data Sets, Metrics, and Training Details

In this section, we initially stress our method in order to show its capabilities, advantages, and major points of possible future work. Thus, we initially evaluate each part of the CNN (which feature extractor is the best), in which learning mechanism produces better results and parts of the scene our model fail to produce accurate results. Furthermore, we also evaluate the refinement methodology by showing the best interpolation function and how it can benefit the overall method. Moreover, we compare our optimal method with respect to other similar counterparts in both aerial and driving environments.

The main goal of our system is to produce accurate depth maps from images captured in forests and open field-like environments by a low-altitude flying drone. The depths will be further used to properly capture and understand the environment and for generating a 3-D mapping and localization. In order to achieve this goal by supervised learning mechanisms (such as in the case of this work), a very accurate GT has to be generated. Most of the current depth perception data sets use real-life images, generating their GTs either by using LiDAR sensors or structured light. Unfortunately, there are no benchmarks for low-altitude aerial scenarios, and the aforementioned techniques cannot be properly applied to aerial images for building a real-life data set. Thus, the only option for us is to move toward synthetic images generated by software tools. Similar depth estimation approaches use AirSim [36], a tool built in Unreal Studio. The tool offers the option to generate depth frames in a variety of scenarios, but the images seem rather unrealistic, without proper textures and illumination conditions. The best alternative that we found was MidAir [66], which introduces a data set for low-altitude UAV flights, with multiple trajectories, in different climates, seasons, and weather conditions. MidAir offers GTs for many visual recognition tasks, such as segmentation, stereo reconstruction, and MDE. The data set provides around 400k individual frames, the quality of the images being superior to any counterpart. For training, we randomly chose 8000 different images, while the other 2000 were selected for evaluation and 2000 for testing. The images have a field of view of 90° and are originally of size 1024×1024 , but we decreased their resolution to half (512×512 px) for computational performance reasons. The GTs have 16 bits/pixel (saved as the *float16* type), with a range between 1 and 1250 m. We further reduced the range to 1–150 m for most of our experiments, all the points beyond this cutoff being estimated at 150 m.

In terms of metrics, we used the MDE metrics generally used in data sets, such as Kitti and NYU. We highlight the

metrics that we used here, but their full description can be found in the corresponding benchmarks [1], [3].

- 1) $\delta 1$, $\delta 2$, and $\delta 3$ represent the ratios between the output and the target values, with a predefined threshold (this should be high, for better accuracy).
- 2) RMSE and $\log_{10}$ are absolute errors, representing the average metric errors either in square or in log form (lower means better);
- 3) absErrorRel is a metric that computes percentages relative to the number of evaluated GT pixels (also here lower means better).

We used our own PyTorch implementations for all the variants of the CNN algorithm. The training is done on 2 Tesla V100-PCIE GPUs, using a batch size of 12. The deployment of the network will be next done on a smaller machine that has a smaller GPU stationed near the UAV (but the network will not run directly on a drone). Considering this, we aim to find the model with the best accuracy while also keeping the resources as limited as possible.

For training (in the case of ordinal regression), we used a learning mechanism based on polynomial decay, with a starting learning rate of 10^{-5} , a momentum of 0.9, and decay of 0.0005. The network converges in about 100 epochs. For the combined learning (ordinal regression + classification), we used the same methodology, but we gradually included the cross-entropy loss, starting from epoch 20. In postprocessing, the training (for the interpolation function) is also done on the GPU, the GA running for 1000 iterations/generations.

In order to obtain an optimal interpolation function for postprocessing, we generate a histogram-like fractional GT set (similar to the GT in Fig. 6, in orange). For this, we randomly extract points from all MidAir images to cover the entire $[-\text{samp}_f/2, \dots, \text{samp}_f/2]$ interval. For simplification, we basically reduce the pixel selection only to the fractional part, projected in the interval $[-0.5, \dots, 0.5]$. We will discretize this interval into 21 subclasses, resulting in subintervals of size 0.05. Finally, the function that we obtain will be scaled according to the samp_f [as mentioned in (15)].

B. CNN Choice

In this part, we thoroughly evaluate the CNN methods that we proposed. Thus, we support the design decisions that we took by showing the optimal choices in terms of feature extractor and learning.

1) *Feature Extractor*: In this section, we compare multiple feature extractors in order to capture the method that gives the best results in terms of accuracy by also keeping the resource consumption as low as possible. We evaluate the four aforementioned well-known feature extractors, ResNet (we test the 50 and 101 variants), Inception, Xception, and Darknet-53, and assess their capabilities in relation to depth perception. We evaluate here the accuracy (root MSE gives an absolute error in metrics), the number of used parameters (for the entire network), and the speed that they can obtain. The model uses 150 depth classes, using a log-based discretization, with a maximum depth allowed of 150 m. No postprocessing is done in this case.

TABLE I
EVALUATION OF VARIOUS FEATURE EXTRACTORS
IN THE CASE OF MDE TASK

Method	RMSE	Parameter No	Runtime
ResNet-50	11.42 m	108 M	150 ms
ResNet-101	9.48 m	186 M	175 ms
Inception	12.04 m	107 M	160 ms
Xception	11.87 m	105 M	155 ms
Darknet-53	9.36 m	128 M	140 ms

It can be seen that Darknet is the best option for us since it obtains results similar to ResNet-101 (even a little bit better), but it uses fewer parameters. Furthermore, although it provides very accurate results, the inference runtime is lower than all the counterparts (only 140 ms on the aforementioned GPU). Numerical results are presented in Table I.

2) *Loss*: We also evaluate the aforementioned losses: a regular classification, an ordinal regression, and the combination between the two. For the multitask loss, we evaluate two variants: one pretrained on Kitti [1] and one without pretraining. For completeness, we also include a regular regression loss, based on MSE and a multitask loss by combining the regular regression and classification tasks. The improvements given by the novel loss can be seen in Table II, while visual results (see Fig. 7) further highlight the importance of the multitask loss. The novel learning mechanism clearly acknowledges for all the classes, predicting the depth around edges more accurately while also moving the objects situated beyond the cutoff max_depth to the *others* class. The pretraining on the Kitti data set also seems to improve the quality of our network. Fig. 7 presents both the depth maps obtained by ordinal regression and the multitask loss (columns 3 and 4—using the *jet* colormap) and the error maps (last two rows) for the two methods. Here, a color closer to blue means a smaller error. The errors that our novel method alleviates are clearly seen in columns 3 and 5 and are presented especially in the last class (for *others*) and for points at larger distances.

C. Refinement Evaluation

Function generation for depth points redistribution has been previously studied in the context of subpixel refinement for stereo reconstruction. Thus, we use such a function as possible alternatives to our method. The tested interpolation functions are parabola and equiangular from [56], SinFit [58], and the function that we generated with our methodology. As evaluation metrics, we use three of the depth estimation metrics proposed on the Kitti benchmark: the mean absolute error relative to the number of pixels (absErrRel) and the metric error of RMSE. For this case, we use the CNN implementation for 150 depth intervals, with a maximum allowed value of 80 m.

As presented in Table III, the best postprocessing results are obtained when using the function we proposed. It can be seen that functions that usually perform well in the case of stereo subpixel correction perform decently in this case, which means that any sort of known point redistribution works fine for MDE methods. The second best correction method is the parabola,

TABLE II
PERFORMANCE OBTAINED USING DIFFERENT TYPES OF LOSSES ON AERIAL IMAGES FROM MIDAIR DATA SET

Loss type	$\delta 1(< 1.25)$	$\delta 2(< 1.25^2)$	$\delta 3(< 1.25^3)$	RMSE	$\log_{10}$	absErrorRel
Regular Regression	0.562	0.863	0.932	22.31	0.139	0.517
Regular Classification	0.621	0.886	0.942	20.31	0.132	0.437
Ordinal regression	0.716	0.921	0.986	16.54	0.078	0.371
Multi-task (regression + classification)	0.671	0.901	0.943	19.53	0.122	0.362
Multi-task (proposed) – pretrained	0.851	0.971	0.989	9.36	0.063	0.136
Multi-task (proposed) – from scratch	0.802	0.961	0.982	10.11	0.089	0.171

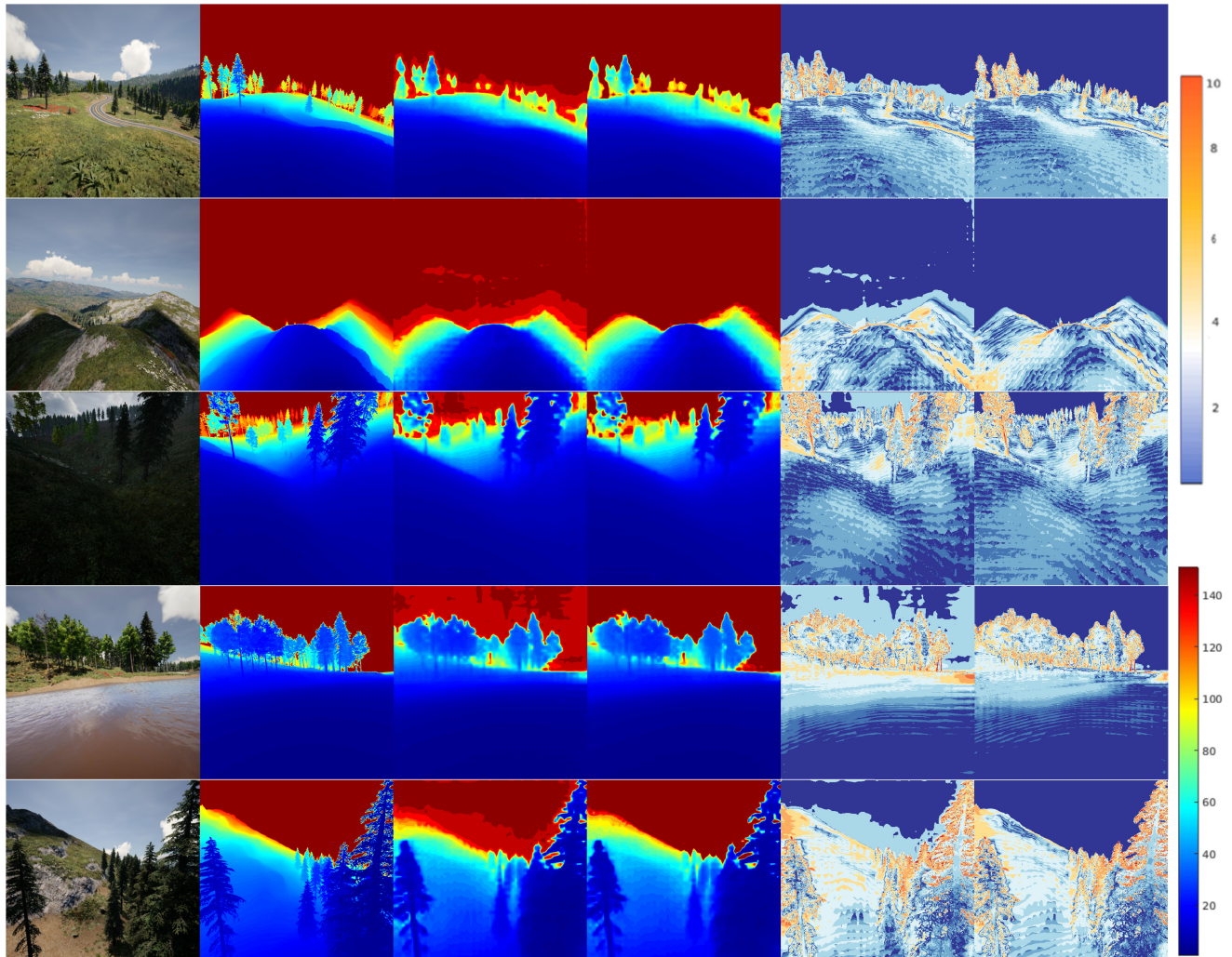


Fig. 7. Results obtained on MidAir images: input image, GT image, depth obtained using ordinal regression, depth image obtained using the proposed multitask loss, error image for the ordinal regression, and error image for the proposed approach. (Right) Scales for error (Top) and depth (Bottom)—colors correspond to meters.

which redistributes points almost similar to our method [if we were to plot the function, its form will be quite similar to the form of our function presented in (18)]. We must note that very few resources are required for implementing the aforementioned function (several additional computations/pixel). This translates to an additional computational time of less than 1 ms.

D. Long-Range Accuracy Improvements

In this part, we examine the results obtained by our method when focusing on different parts of the scene. We are particularly interested in dealing with objects at large distances and

check how the postprocessing might help in those areas. This part is also significant if we use the log-space discretization for the CNN since the approach overlooks objects on the far side. Thus, we initially evaluate our CNN (without the

TABLE III
PERFORMANCE OBTAINED WITH DIFFERENT INTERPOLATION FUNCTIONS

Function	absErrorRel	RMSE
Without	0.136	9.36
Parabola	0.118	8.78
Equiangular	0.124	8.98
SinFit	0.125	9.06
Proposed	0.116	8.58

TABLE IV
PERFORMANCE OBTAINED FOR PIXELS AT VARIOUS DISTANCES

Method	Range	$\delta 1(< 1.25)$	$\delta 2(< 1.25^2)$	$\delta 3(< 1.25^3)$	RMSE	$\log_{10}$	absErrorRel
Ordinal regression	0 - 150	0.716	0.921	0.986	16.54	0.078	0.241
Multi-task	0 - 150	0.851	0.971	0.989	9.36	0.063	0.136
Multi-task	0 - 30	0.912	0.98	0.993	7.54	0.039	0.102
Multi-task	80 - 150	0.801	0.955	0.982	14.86	0.064	0.147
Multi-task	80 - 149	0.878	0.956	0.971	19.57	0.077	0.159
Multi-task (+Refinement)	0 - 30	0.941	0.982	0.993	7.34	0.039	0.099
Multi-task (+Refinement)	80 - 150	0.848	0.969	0.982	13.67	0.068	0.14

refinement) and then introduce the interpolation function (second part). We evaluate the accuracy of points with depth class values smaller than 100 (range 0–30 m when the log-space discretization is used) and points at long distances in range 80–150 m (i.e., classes from 130 to 150) when the maximum depth allowed is 150 m and 150 depth classes have been used. We have also tested how our method works for objects in classes 80–149 (without accounting for points labeled in the *others* class, beyond the cutoff).

The first observation is that the CNN behaves very well for near-side objects (due to the very large number of classes pertaining to that area) while performing reasonably well for objects at large distances. Second, we must note that our corrected depth map obtains improved very good results for objects at a large distance while not affecting the near-range accuracies showing that the correction has limited effectiveness for objects on the near side. Numerical results can be seen in Table IV.

E. Sampling Factor Variation

We are interested in seeing how our method behaves when we vary the number of classes: this gives the way in which we discretize the depth space. We test here multiple CNN models, which corresponds to the following number of depth classes/intervals: 160, 80, 40, 20, and 10. We have chosen these values when the max depth value is set to 160. Thus, we use five possible sampling factors: 1, 2, 4, 8, and 16. We test two options: without any postprocessing and corrected using the proposed interpolation function. The first important aspect that we observe is that runtime decreases almost exponentially with depth class reduction, until the feature extractor part becomes the critical part in time consumption. Error, on the other hand, has a much smaller increment. The results can be seen in Fig. 8. It can be also seen that decent performance is obtained when the sampling factors are smaller than 8 (more than 20 depth classes are required). Another important observation is that postprocessing becomes more relevant when the sampling factor is larger (its burden related to depth distribution becomes more relevant).

F. Comparison With State-of-the-Art Methods

In this section, we compare our proposed method (with and without postprocessing) with other MDE counterparts, generally regarded in the literature as the most effective for driving and indoor scenarios (see Table V). Thus, we used the code given by the authors for training in the case of Eigen *et al.* [38], VNL [26], and BTS [39], while we implemented

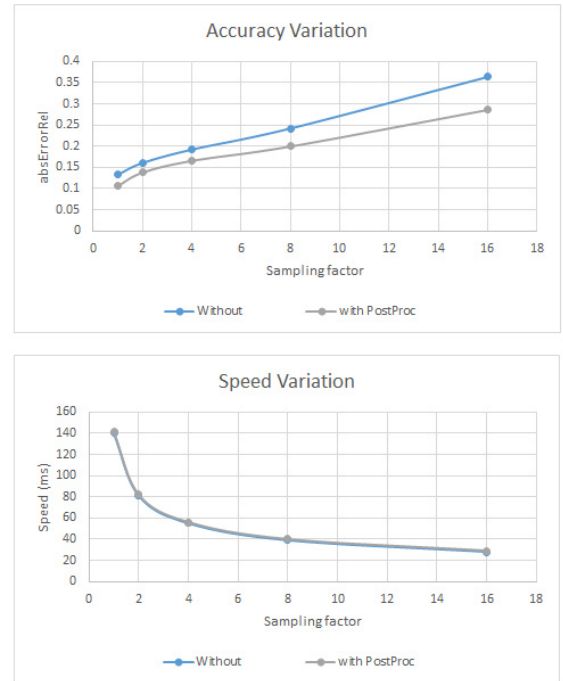


Fig. 8. Accuracy versus speed variation for different sampling factors.

DORN [27] from scratch (the authors only offer the inference code). We used two different DORN models: one based on ResNet-101 and one based on ResNet-50. We also include here the results given by the authors of the MidAir data set [66] that includes, in their paper, some results (only for RMSE and some relative metric—we are not sure if this error is precisely absErrorRel since they do not specify this) obtained by using the Eigen *et al.*'s [38] method. The results prove that our proposed method obtains the best results for the aerial scenario. VNL and BTS perform decently for most of the metrics, but their error (RMSE) is very large, which makes them less appealing for the proposed task. DORN encounters the same difficulties as our ordinal regression-based model (as it can be seen in column 3 in Fig. 7) for objects at large distances, so it is also less useful.

VI. EVALUATION IN REAL-LIFE SCENARIOS

A. UAV System and Image Generation

The most important part of evaluating such a system is checking its results in real-life scenarios, using a real drone equipped with a specific camera, which provides images, while our system generates a depth map of the scene from them.

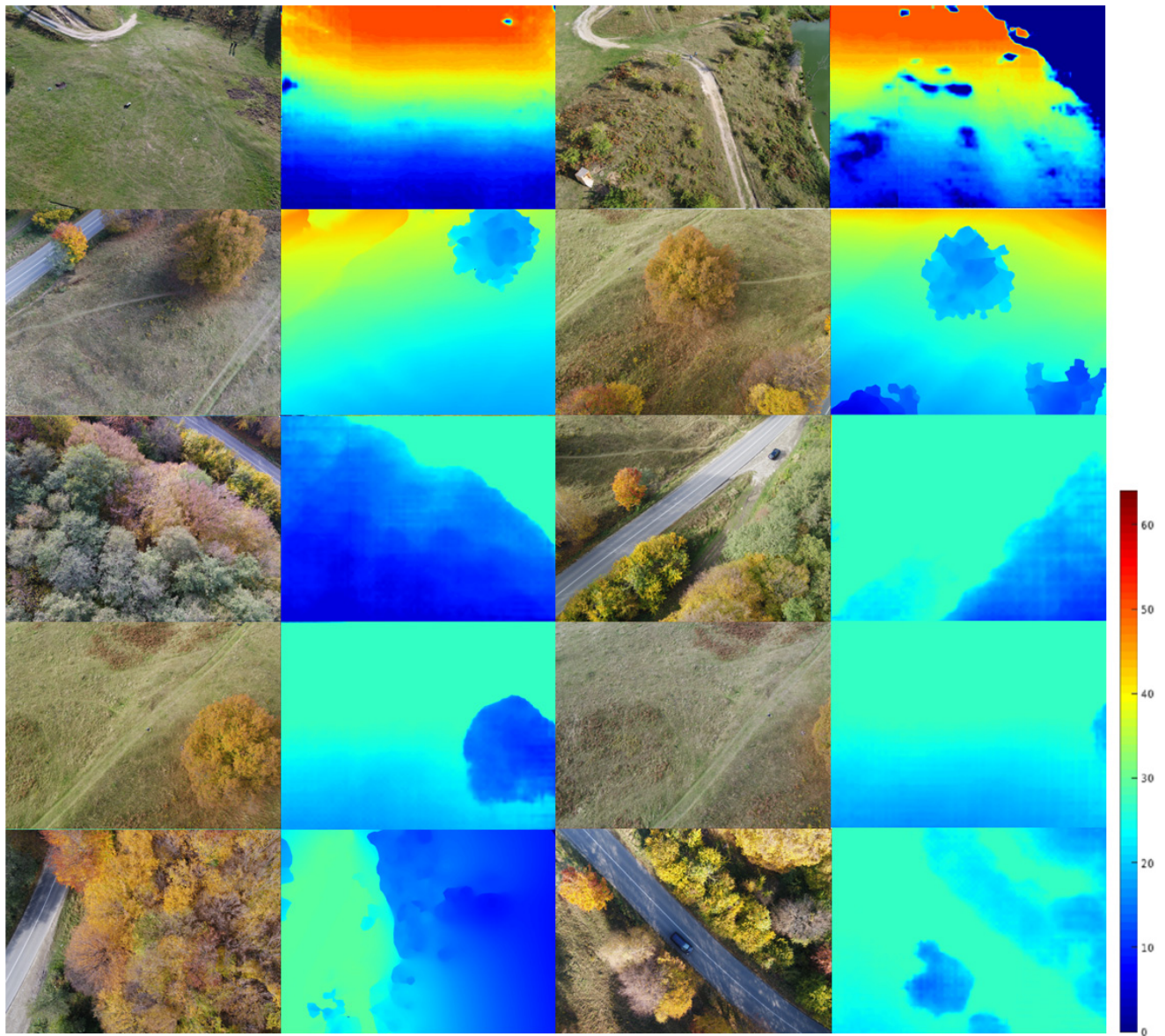


Fig. 9. Results on real images captured from a drone: First two rows: field images at 50° – 70° . Middle row: forest images at 50° – 70° . Last two rows: field and forest nadir images. (Right) Scale for depth image (colors are meters).

For these experiments, we used a DJI Matrice 210 RTK V2 unmanned aerial system equipped with a DJI Zenmuse X5s camera system capable of producing both videos (at 30 frames/s) or images (at 60 frames/s) with high sensitivity and a low noise. The images are captured with a sensor size of 13×17.3 mm, and the focal length of the camera is set at 15 mm, while the pitch angle is set according to each particular scenario. The drone was used in several different scenarios (field-like and forest-like), flying at different altitudes, using real-time kinematic (RTK) positioning and a clearly established base station. For all the scenarios, we used the images (not the videos) as input for our system. All processing was done offline. In all of the presented scenarios, the drone surveyed the environment at a predefined height ranging from 20 to 30 m (with respect to the base station).

The system was trained using the MidAir images and slightly fine-tuned (for 20 epochs) using several GT depth key

points (generated by using the Open Drone Maps software and then projected into the image frames). Each of the original images (5280×3956 px) is downsampled and then separated into two squared images (of 1024×1024 resolution, to resemble the training format). The depth maps are then recombined (with a small overlap) and upsampled to the input resolution.

B. Real Images From Camera With Medium-Large Pitch Angle (50° – 70°)

Since no benchmark dealing with depth perception from real-life images exists (very hard to generate GTs), the only way in which we can evaluate our method is visual. Therefore, Fig. 9 depicts the results that we obtained when applying our MDE system over images captured in real-life scenarios by a drone. For completeness, we selected a variety of scenarios, from different environments.

TABLE V
EVALUATION OF OUR METHOD IN COMPARISON WITH STATE-OF-THE-ART MDE APPROACHES

Method	$\delta 1(< 1.25)$	$\delta 2(< 1.25^2)$	$\delta 3(< 1.25^3)$	RMSE	$\log_{10}$	absErrorRel
DORN (ResNet-101) [27]	0.716	0.919	0.984	16.78	0.103	0.392
DORN (ResNet-50) [27]	0.682	0.905	0.978	17.68	0.123	0.515
Eigen [38]	0.692	0.899	0.967	17.15	0.270	0.690
Eigen [38] in MidAir [66]				17.022		0.887
VNL [26]	0.806	0.957	0.979	13.51	0.086	0.259
BTS [39]	0.816	0.957	0.981	12.48	0.081	0.241
Proposed (-ref)	0.851	0.971	0.989	9.36	0.063	0.136
Proposed (+ref)	0.881	0.978	0.991	8.58	0.052	0.116

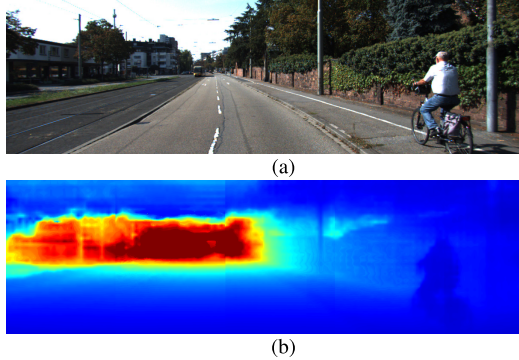


Fig. 10. Results obtained with our depth generation approach for Kitti images. (a) RGB image. (b) Obtained depth.

First, we will study the behavior of our method when applied to images captured from a camera mounted on the UAV, setting a medium/large pitch angle (50° – 70°). We evaluated the system in the following scenarios: 1) a countryside field with a road, a lake, and some small vegetation; 2) a different field that contains a road and some large trees; and 3) a forest that contains multiple types of trees. All images in these scenarios are obtained during the autumn season. The input images and their corresponding depth maps can be seen on the first three rows in Fig. 9. Since the system provides depth points in the camera coordinate system, the results can be easily understood (closer points to the camera will have a smaller depth). As seen in the images and the attached scale, depths vary between 10 (obtained especially for the forest canopy) and 60 m (for objects at a larger distance, as shown in images with a smaller pitch, on the first row). We observe that the obtained results are generally good, the main problems being caused by water and shadowed regions (this can be mainly seen on the first row, right).

C. Nadir Images

The next part of the evaluation deals with computing the depth from Nadir (90° vertical pitch) images. This is the most difficult scenario for our system since the unstructured ordering of the objects creates many ambiguities for the CNN. However, good results have been obtained by our method, especially due to the additional fine-tuning. These can be seen in the last two rows of Fig. 9, which also depicts the results obtained by our method on images captured in: 1) a field environment and 2) a forest environment. The accuracy of our method can be especially understood when comparing the distance toward the forest canopy with respect to the ground/road in its proximity.

D. Evaluation on Driving Data Sets

Finally, we show that our method can work also in other scenarios, and we highlight the results that it obtains for driving scenes. For this case, we use images from the Kitti [1] data set. The validity of our method can be visually observed in Fig. 10, which depicts the results obtained by our refined MDE method. The depth map is depicted in Fig. 10(b), the color codes for the map being similar to the ones presented in the previous figure. It can be visually seen that all pixels are accurately computed, the depth map in the entire scene being reliable.

VII. CONCLUSION

In this work, we have presented a novel method to generate a depth map of aerial-captured environments using MDE. The method is based on a novel CNN that is particularly tailored to deal with the challenges posed by unstructured aerial environments. The architecture combines an optimal feature extractor with a novel upsampling subnetwork and a new loss that uses both ordinal regression and classification to model the learning process. A novel CNN layer that transforms the ordinal regression softmax into a classification-based one facilitates the training while also enabling subpixel post-processing to further refine the depth map. The refinement process uses GAs to optimally generate interpolation functions that redistribute depth points along with the entire interval. Our method is validated on synthetically generated and real aerial images showing it to perform well in a large variety of scenarios.

We plan to extend our work by increasing the reliability of our system in other real-life aerial scenarios. Furthermore, we plan to obtain a further reduction of the computation burden required by such a system in order to enable full onboard processing.

REFERENCES

- [1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 3354–3361.
- [2] D. Scharstein and R. Szeliski, "A taxonomy and evaluation of dense two-frame stereo correspondence algorithms," *Int. J. Comput. Vis.*, vol. 47, nos. 1–3, pp. 7–42, Apr. 2002. [Online]. Available: <http://research.microsoft.com/apps/pubs/default.aspx?id=64200>
- [3] P. K. Nathan Silberman, D. Hoiem, and R. Fergus, "Indoor segmentation and support inference from RGBD images," in *Proc. ECCV*, 2012, pp. 746–760.
- [4] D. Scharstein, R. Szeliski, and R. Zabih, "A taxonomy and evaluation of dense two-frame stereo correspondence algorithms," in *Proc. IEEE Workshop Stereo Multi-Baseline Vis. (SMBV)*, 2001, pp. 131–140.

- [5] H. Hirschmuller, "Stereo processing by semiglobal matching and mutual information," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 30, no. 2, pp. 328–341, Feb. 2008.
- [6] V.-C. Miclea and S. Nedeveschi, "Real-time semantic segmentation-based stereo reconstruction," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 4, pp. 1514–1524, Apr. 2020.
- [7] M. Mahato, S. Gedam, J. Joglekar, and K. M. Buddhiraju, "Dense stereo matching based on multiobjective fitness function—A genetic algorithm optimization approach for stereo correspondence," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 6, pp. 3341–3353, Jun. 2019.
- [8] D. Steinhauser, O. Ruepp, and D. Burschka, "Motion segmentation and scene classification from 3D LIDAR data," in *Proc. IEEE Intell. Vehicles Symp.*, Jun. 2008, pp. 398–403.
- [9] W. Maddern and P. Newman, "Real-time probabilistic fusion of sparse 3D LIDAR and dense stereo," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2016, pp. 2181–2188.
- [10] O. Ozyesil, V. Voroninski, R. Basri, and A. Singer, "A survey of structure from motion," 2017, *arXiv:1701.08493*. [Online]. Available: <http://arxiv.org/abs/1701.08493>
- [11] A. Bhoi, "Monocular depth estimation: A survey," 2019, *arXiv:1901.09402*. [Online]. Available: <http://arxiv.org/abs/1901.09402>
- [12] F. Liu, C. Shen, G. Lin, and I. Reid, "Learning depth from single monocular images using deep convolutional neural fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 10, pp. 2024–2039, Oct. 2016, doi: [10.1109/TPAMI.2015.2505283](https://doi.org/10.1109/TPAMI.2015.2505283).
- [13] A. Harikumar, F. Bovolo, and L. Bruzzone, "A local projection-based approach to individual tree detection and 3-D crown delineation in multistoried coniferous forests using high-density airborne LiDAR data," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1168–1182, Feb. 2019.
- [14] J. Tu, J. Yao, L. Li, W. Zhao, and B. Xiang, "Extraction of street pole-like objects based on plane filtering from mobile LiDAR data," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 749–768, Jan. 2021.
- [15] A. Harikumar, C. Paris, F. Bovolo, and L. Bruzzone, "A crown quantization-based approach to tree-species classification using high-density airborne laser scanning data," *IEEE Trans. Geosci. Remote Sens.*, early access, Aug. 10, 2021, doi: [10.1109/TGRS.2020.3012343](https://doi.org/10.1109/TGRS.2020.3012343).
- [16] J. Williams *et al.*, "3D segmentation of trees through a flexible multiclass graph cut algorithm," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 2, pp. 754–776, Feb. 2020.
- [17] X. Du and A. Zare, "Multiresolution multimodal sensor fusion for remote sensing data with label uncertainty," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2755–2769, Apr. 2020.
- [18] O. Kechagias-Stamatis and N. Aouf, "A new passive 3-D automatic target recognition architecture for aerial platforms," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 1, pp. 406–415, Jan. 2019.
- [19] J. Li, Y. Liu, S. Du, P. Wu, and Z. Xu, "Hierarchical and adaptive phase correlation for precise disparity estimation of UAV images," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 12, pp. 7092–7104, Dec. 2016.
- [20] J. Liu, L. Zhang, Z. Wang, and R. Wang, "Dense stereo matching strategy for oblique images that considers the plane directions in urban areas," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 7, pp. 5109–5116, Jul. 2020.
- [21] F. Valenti, D. Giaquinto, L. Musto, A. Zinelli, M. Bertozzi, and A. Broggi, "Enabling computer vision-based autonomous navigation for unmanned aerial vehicles in cluttered GPS-denied environments," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Nov. 2018, pp. 3886–3891.
- [22] B. Brzozowski and N. Szymanek, "Stereo vision module for UAV navigation system," in *Proc. 5th IEEE Int. Workshop Metro. Aerosp. (MetroAeroSpace)*, Jun. 2018, pp. 2422–2425.
- [23] Z. Rao, M. He, Z. Zhu, Y. Dai, and R. He, "Bidirectional guided attention network for 3-D semantic detection of remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, early access, Oct. 22, 2020, doi: [10.1109/TGRS.2020.3029527](https://doi.org/10.1109/TGRS.2020.3029527).
- [24] H. Guan *et al.*, "A novel framework to automatically fuse multiplatform LiDAR data in forest environments based on tree locations," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 3, pp. 2165–2177, Mar. 2020.
- [25] T. C. Bybee and S. E. Budge, "Method for 3-D scene reconstruction using fused LiDAR and imagery from a texel camera," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 11, pp. 8879–8889, Nov. 2019.
- [26] W. Yin, Y. Liu, C. Shen, and Y. Yan, "Enforcing geometric constraints of virtual normal for depth prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 5684–5693.
- [27] H. Fu, M. Gong, C. Wang, K. Batmanghelich, and D. Tao, "Deep ordinal regression network for monocular depth estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2002–2011.
- [28] N. Thanomsing, N. Boonruam, P. Sirisawat, W. Nonsakhoo, and S. Saiyod, "Landscape analysis system using 3D stereoscopic for drone," in *Proc. IEEE 13th Int. Colloq. Signal Process. Appl. (CSPA)*, Mar. 2017, pp. 118–122.
- [29] L. Teixeira, M. R. Oswald, M. Pollefeys, and M. Chli, "Aerial single-view depth completion with image-guided uncertainty estimation," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1055–1062, Apr. 2020.
- [30] M. Nieuwenhuisen, D. Droschel, M. Beul, and S. Behnke, "Obstacle detection and navigation planning for autonomous micro aerial vehicles," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, May 2014, pp. 1040–1047.
- [31] L. von Stumberg, V. Usenko, J. Engel, J. Stuckler, and D. Cremers, "From monocular SLAM to autonomous drone exploration," in *Proc. Eur. Conf. Mobile Robots (ECMR)*, Sep. 2017, pp. 1–8.
- [32] X. Yang *et al.*, "Fast depth prediction and obstacle avoidance on a monocular drone using probabilistic convolutional neural network," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 156–167, Jan. 2021.
- [33] K. Julian, J. Mern, and R. Tompa, "UAV depth perception from visual images using a deep convolutional neural network," Stanford Univ., Stanford, CA, USA, Tech. Rep.
- [34] A. Marcu, D. Costea, V. Licaret, M. Pirvu, M. Leordeanu, and E. Slusanschi, "SafeUAV: Learning to estimate depth and safe landing areas for UAVs from synthetic data," in *Proc. Eur. Conf. Comput. Vis. (ECCV) Workshops*, 2018, pp. 43–58.
- [35] Z. Zhang, M. Xiong, and H. Xiong, "Monocular depth estimation for UAV obstacle avoidance," in *Proc. 4th Int. Conf. Cloud Comput. Internet Things (CCIoT)*, Dec. 2019, pp. 43–47.
- [36] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," 2017, *arXiv:1705.05065*. [Online]. Available: <http://arxiv.org/abs/1705.05065>
- [37] Y. Tian, X. Li, K. Wang, and F. Wang, "Training and testing object detectors with virtual images," in *IEEE/CAA J. Automatica Sinica*, vol. 5, no. 2, pp. 539–546, Mar. 2018, doi: [10.1109/JAS.2017.7510841](https://doi.org/10.1109/JAS.2017.7510841).
- [38] D. Eigen, C. Puhrsch, and R. Fergus, "Depth map prediction from a single image using a multi-scale deep network," 2014, *arXiv:1406.2283*. [Online]. Available: <http://arxiv.org/abs/1406.2283>
- [39] J. Han Lee, M.-K. Han, D. Wook Ko, and I. Hong Suh, "From big to small: Multi-scale local planar guidance for monocular depth estimation," 2019, *arXiv:1907.10326*. [Online]. Available: <http://arxiv.org/abs/1907.10326>
- [40] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," 2016, *arXiv:1612.03144*. [Online]. Available: <http://arxiv.org/abs/1612.03144>
- [41] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," 2015, *arXiv:1511.07122*. [Online]. Available: <http://arxiv.org/abs/1511.07122>
- [42] S. Kong and C. Fowlkes, "Pixel-wise attentional gating for parsimonious pixel labeling," 2018, *arXiv:1805.01556*. [Online]. Available: <http://arxiv.org/abs/1805.01556>
- [43] R. Li, K. Xian, C. Shen, Z. Cao, H. Lu, and L. Hang, "Deep attention-based classification network for robust depth prediction," 2018, *arXiv:1807.03959*. [Online]. Available: <http://arxiv.org/abs/1807.03959>
- [44] Y. Cao, Z. Wu, and C. Shen, "Estimating depth from monocular images as classification using deep fully convolutional residual networks," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 28, no. 11, pp. 3174–3182, Nov. 2018.
- [45] R. Diaz and A. Marathe, "Soft labels for ordinal regression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4733–4742.
- [46] Y. Kuznetsov, J. Stuckler, and B. Leibe, "Semi-supervised deep learning for monocular depth map prediction," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2215–2223.
- [47] A. Jahani Amiri, S. Yan Loo, and H. Zhang, "Semi-supervised monocular depth estimation with left-right consistency using deep neural network," 2019, *arXiv:1905.07542*. [Online]. Available: <http://arxiv.org/abs/1905.07542>
- [48] C. Godard, O. M. Aodha, and G. J. Brostow, "Unsupervised monocular depth estimation with left-right consistency," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 6602–6611.
- [49] F. Tosi, F. Aleotti, M. Poggi, and S. Mattoccia, "Learning monocular depth estimation infusing traditional stereo knowledge," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 9799–9809.

- [50] Y. Luo *et al.*, "Single view stereo matching," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 155–163.
- [51] P. Pinggera, D. Pfeiffer, U. Franke, and R. Mester, "Know your limits: Accuracy of long range stereoscopic object measurements in practice," in *Computer Vision—(ECCV)* (Lecture Notes in Computer Science), vol. 8690, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds. Cham, Switzerland: Springer, 2014, pp. 96–111.
- [52] V.-C. Miclea and S. Nedevschi, "A unified method for improving long-range accuracy of stereo and monocular depth estimation algorithms," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Oct. 2020, pp. 1234–1241.
- [53] Y. Cheng and L. H. Matthies, "Stereovision bias removal by autocorrelation," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, Jan. 2015, pp. 1153–1160.
- [54] V.-C. Miclea, C.-C. Vancea, and S. Nedevschi, "New sub-pixel interpolation functions for accurate real-time stereo-matching algorithms," in *Proc. IEEE Int. Conf. Intell. Comput. Commun. Process. (ICCP)*, Sep. 2015, pp. 173–178.
- [55] V.-C. Miclea and S. Nedevschi, "Semantic segmentation-based stereo reconstruction with statistically improved long range accuracy," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2017, pp. 1795–1802.
- [56] M. Shimizu and M. Okutomi, "Sub-pixel estimation error cancellation on area-based matching," *Int. J. Comput. Vis.*, vol. 63, no. 3, pp. 207–224, 2005.
- [57] C.-C. Vancea, V.-C. Miclea, and S. Nedevschi, "Improving stereo reconstruction by sub-pixel correction using histogram matching," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2016, pp. 335–341.
- [58] I. Haller and S. Nedevschi, "Design of interpolation functions for subpixel-accuracy stereo-vision systems," *IEEE Trans. Image Process.*, vol. 21, no. 2, pp. 889–898, Feb. 2012.
- [59] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [60] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.
- [61] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1800–1807.
- [62] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*. [Online]. Available: <http://arxiv.org/abs/1804.02767>
- [63] M. Menze and A. Geiger, "Object scene flow for autonomous vehicles," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3061–3070.
- [64] M. Cordts *et al.*, "The cityscapes dataset for semantic urban scene understanding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 3213–3223.
- [65] J. Zbontar and Y. LeCun, "Computing the stereo matching cost with a convolutional neural network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1592–1599.
- [66] M. Fonder and M. Van Droogenbroeck, "Mid-air: A multi-modal dataset for extremely low altitude drone flights," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2019, pp. 553–562.



Vlad-Cristian Miclea (Member, IEEE) received the M.S. degree from the École Normale Supérieure de Lyon (ENS), Lyon, France, in 2014. He is pursuing the Ph.D. degree with the Faculty of Automation and Computer Science, Technical University of Cluj-Napoca (TUCN), Cluj-Napoca, Romania.

His research interests include image processing, 3-D vision, and machine learning.



Sergiu Nedevschi (Member, IEEE) received the M.S. and Ph.D. degrees in electrical engineering from the Technical University of Cluj-Napoca (TUCN), Cluj-Napoca, Romania, in 1975 and 1993, respectively.

From 1976 to 1983, he was a Researcher with the Research Institute for Computer Technologies, Cluj-Napoca. In 1983, he joined TUCN, where he was appointed as a Full Professor of computer science in 1998 and founded and since then he has been leading the Image Processing and Pattern Recognition Research Center. From 2000 to 2004, he was the Head of the Computer Science Department, from 2004 to 2012, he was the Dean of the Faculty of Automation and Computer Science, and from 2012 to 2020, he was the Vice-Rector with the Scientific Research of TUCN. He was involved in more than 80 research projects, being the coordinator of 62 of them. His industrial cooperation with important automotive players, such as Volkswagen AG, Robert Bosch GmbH, and SICK AG all from Germany, and research institutes, such as VTT from Finland and INRIA from France, was achieved through funded research projects. He has published more than 400 scientific articles and has edited over 20 volumes, including books and conference proceedings. His research interests include image processing, pattern recognition, computer vision, machine learning, intelligence, and autonomous vehicles.